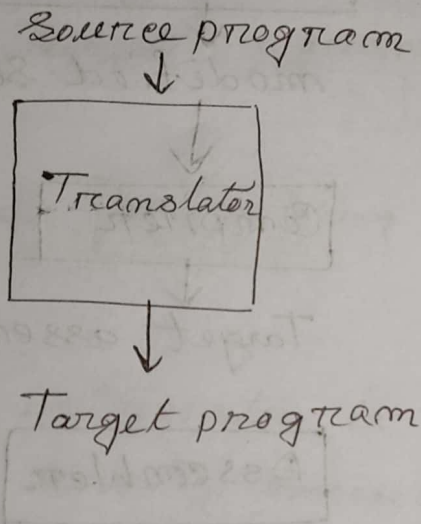


What does language translator means? Give some example of source language, translator and target language.

Translator read source language and translate it into target language. Translator, is also called compiler.



Source program : High level language programming

Ex : 'C' programming language

Translator : Translate the source program

Ex : Compiler, Interpreter.

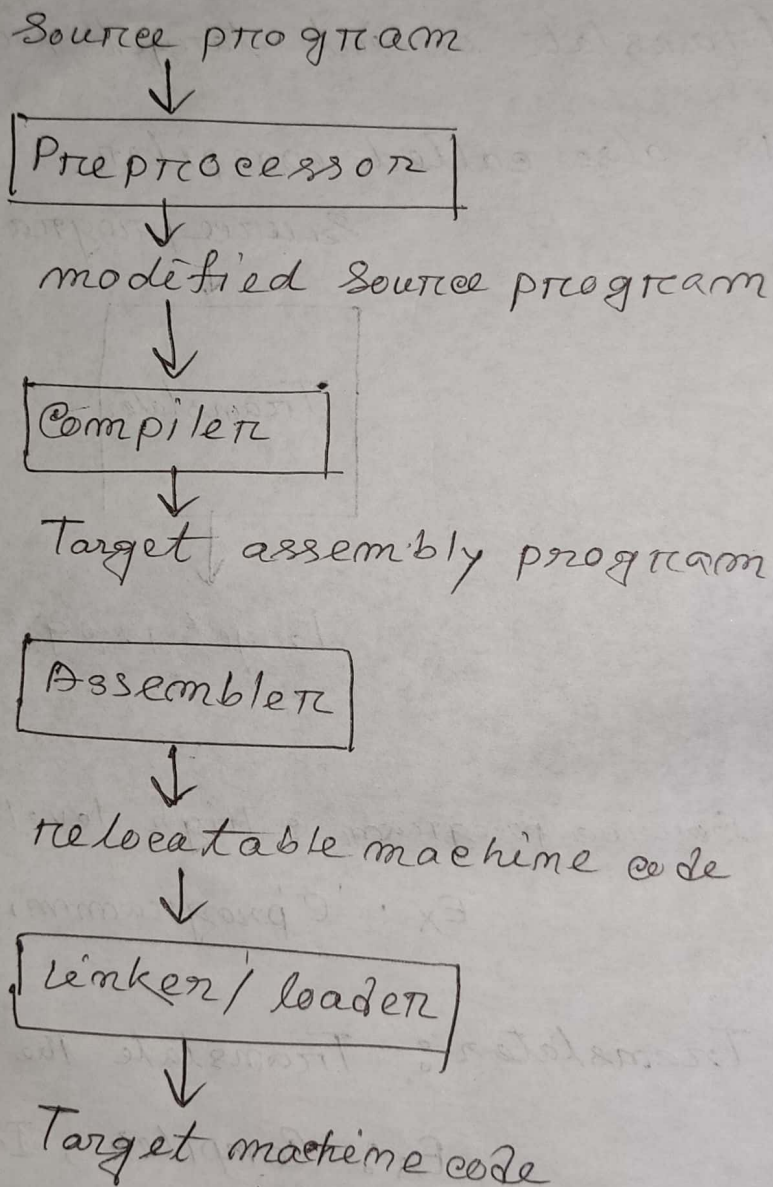
Target program : output of the translation process

Ex : Machine code, Intermediate code.

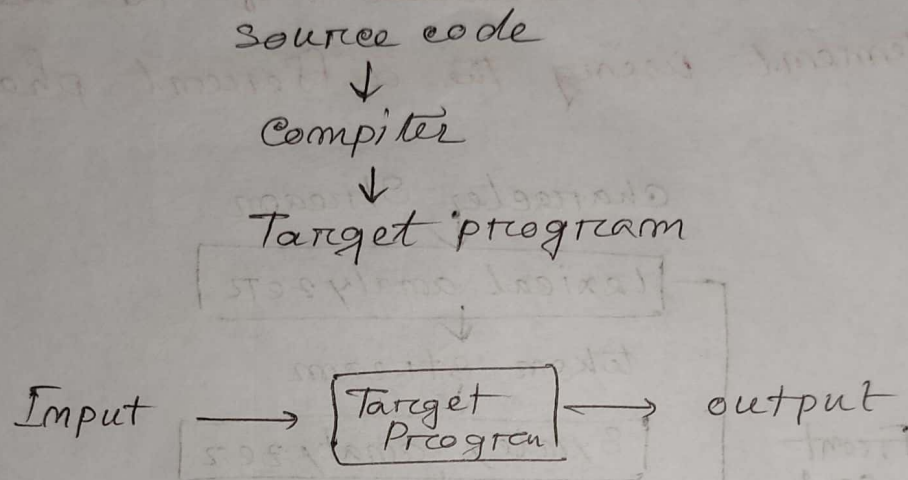
what does compiler mean? How do you distinguish between compiler and interpreter?

Show the steps of language processing system.

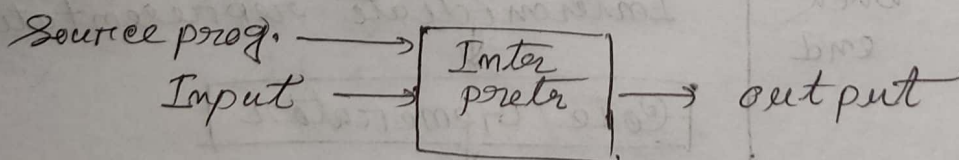
How do you distinguish between compiler and interpreter.



Compiler : Translate source language into target language. Important role of compiler is to report any error of source program during translation process.



Interpreter : Interpreter is a language processor that directly execute instruction written in high level programming language without separate compilation step.



Compiler	Interpreter
reporting error	better error diagnostics
Faster	less Faster

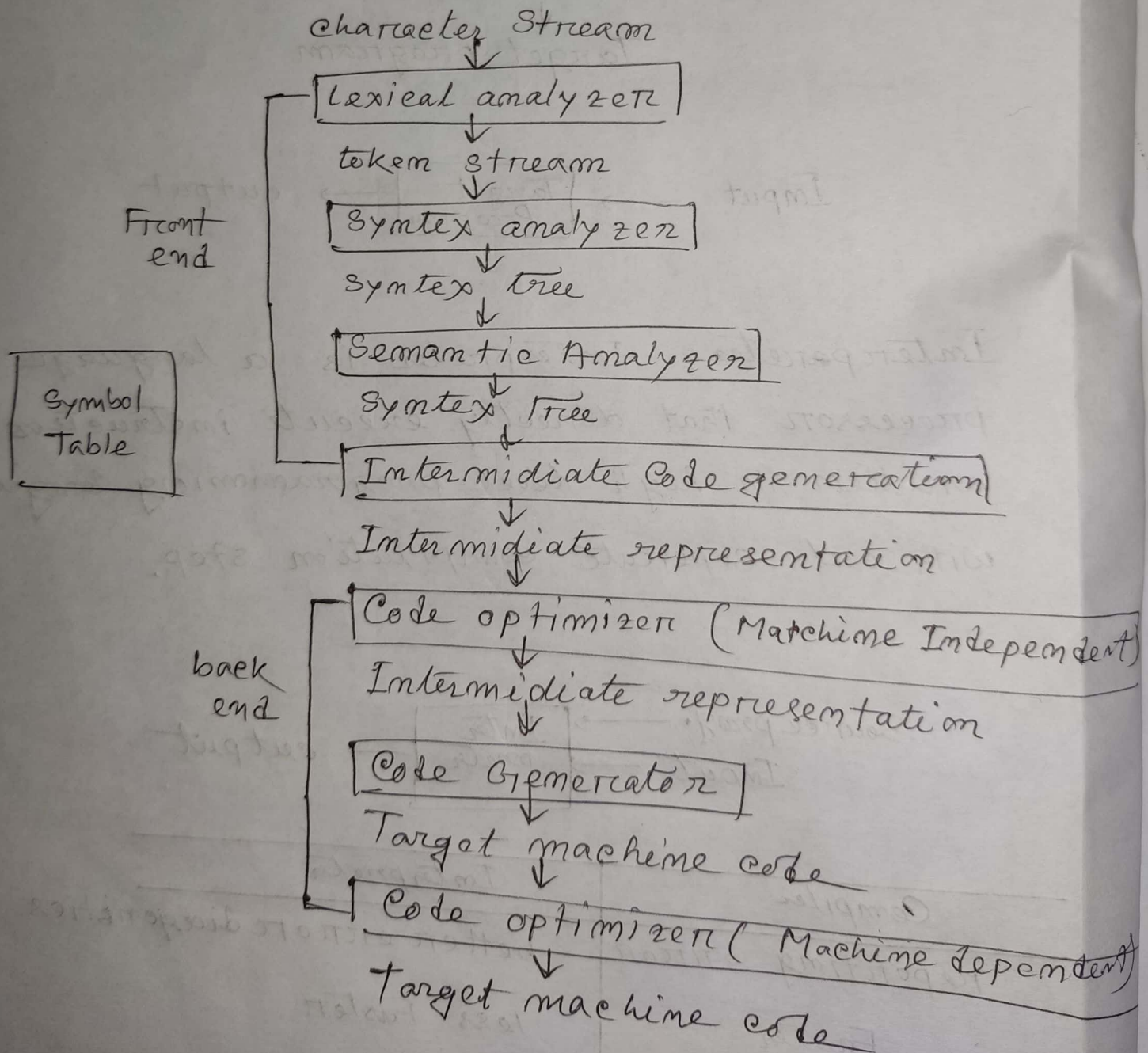
Identify frontend & backend?

Suppose a source program contains the assignment statement
 $\text{statement position} = \text{initial} + \text{rate} * 60$ show representation.

Illustrate the Typical decomposition of a compiler into phases. Suppose a source program contains the assignment statement

$$\text{Position} = \text{initial} + \text{rate} * 60$$

Show the representation of the assignment statement using the different phases in compiler



Position = initial + rate * 60



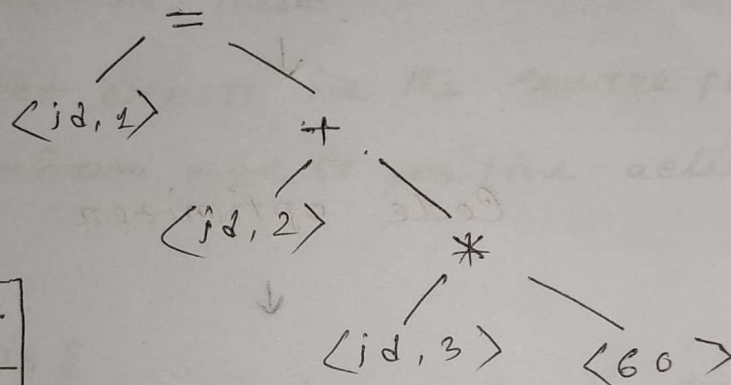
Lexical Analyzer



$\langle id, 1 \rangle$ Position $\langle = \rangle$ $\langle id, 2 \rangle$ $\langle + \rangle$ $\langle id, 3 \rangle$ $\langle * \rangle$ $\langle 60 \rangle$



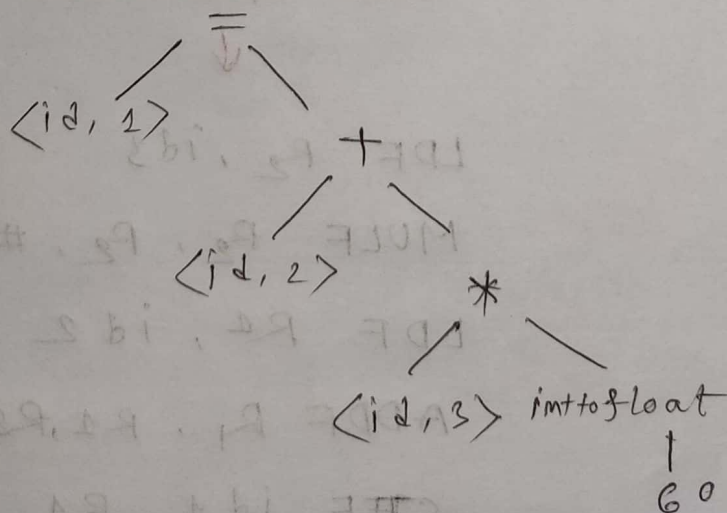
Syntax Analyzer



1	position	- - -
2	initial	- - -
3	rate	- - -
	⋮	

Symbol table

Semantic Analyzer



Intermediate Code generation



$t_1 = \text{int to float}(60);$

$t_2 = \text{id3} * t_1$

$t_3 = \text{id2} + t_2$

$\text{id1} = t_3$



Code optimizer



$t_1 = \text{id3} * 60$

$t_2 = \text{id2} + t_1$



Code Generator



LDF R2, id3

MULF R2, R2, #60.0

LDF R1, id2

ADDF R1, R1, R2

STF id1, R1

Compiler has two part.

1. Analysis part $\rightarrow$ Front end
2. Synthesise part $\rightarrow$ Back end

▣ Analysis:

It break down the source program into smaller pieces and impose grammatical structure on them.

It detect errors in the source program and inform user to positive action.

▣ Synthesis:

Synthesis part construct the desired target program from the independent representation.

Write down the main task of lexical Analyzer.
 Define tokens patterns, and lexemes. Explore
 the token patterns and lexemes from the C statement
 Print ("Total = %d\n", score);

The main task of lexical Analyzer is to read
 the input characters of the source program and
 group them into lexemes.

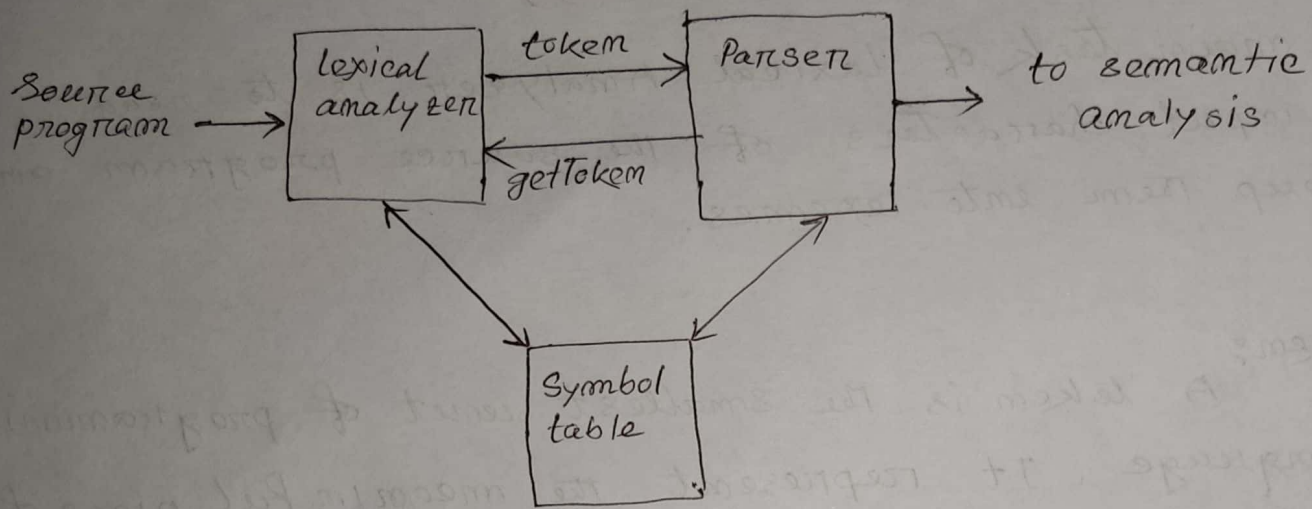
Token: A token is the smallest unit of programming
 language. It represents the meaningful piece of
 information and is often categorized into
 different types of keyword, identifiers, literals
 and operators.

Lexeme: Is a sequence of character in the source
 code that matches the pattern for a particular token

Pattern: Pattern is a rule or specification that
 describes the set of possible lexemes for a particular
 tokens.

Pattern	Token	Lexeme
Printf	Identifier :	Printf $\rightarrow$ identifier lexeme
(	Print, score	($\rightarrow$ left parenthesis lexeme
"Total = %d\n"	Parentheses : (,)	"Total = %d\n" $\rightarrow$ literal
,	String literal :	, $\rightarrow$ comma lexeme
score	"Total = %d\n"	score $\rightarrow$ identifier lexeme
)	Comma : ,	) $\rightarrow$ right parenthesis lexeme
;	Semicolon : ;	; $\rightarrow$ semicolon lexeme

What is the role of lexical analyzer? represent the interaction between lexical analyzer and parser.



lexical analyzer read input character of source program and group them into lexemes and produce output of a sequence of token.

Parser read the token and produce a parse tree. lexical analyzer and parser both interact with the symbol table.

lexical analyzer divided into two process

1. Scanning: deletion of comments

remove white space

WS $\rightarrow$ (Newline, blank, Tab)

2. lexical analysis:

Produce token

Distinguish the difference between Kleen closure of L and positive closure of L . How to build a regular expression using basis and induction.

Kleen closure:

Kleen closure of language L denoted L^* . The strings get concatenating (L^0) L zero's and more.

$$\text{Kleen closure of } L^* = \bigcup_{i=0}^{\infty} L^i$$
$$\therefore L^+ \cup \{e\}$$

Positive closure:

Positive closure of language L denoted L^+ . The strings. The strings get concatenation without the term L^0 .

$$\text{Positive closure of } L = L^+ = \bigcup_{i=1}^{\infty} L^i$$

$$\therefore L^1 \cup L^2 \cup L^3 \dots L^n$$

Regular expression is a concise pattern matching notation used to searching and manipulating text.

Basis: There are two rules for the basis

1. ϵ is the regular expression

$$L(\epsilon) = \{ \epsilon \}$$

↑
member is empty string.

2. a is the ~~at~~ symbol in Σ , then a is a regular expression

$$L(a) = \{ a \}$$

↑
language with one string

Induction: Four part of induction.

1. $(r)|(s)$ is a regular expression denoting the language $L(r) \cup L(s)$

2. $(r)(s)$ is a " " " " $L(r) L(s)$

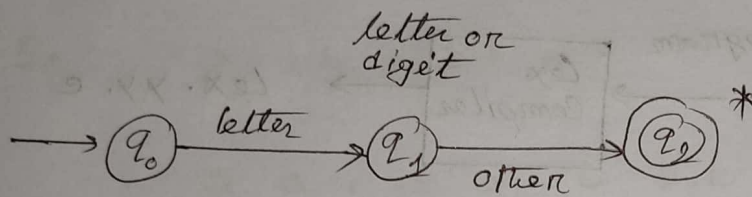
3. $(r)^*$ is " " " " $(L(r))^*$

4. (r) is the regular expression denoting the language $L(r)$.

How to recognize reserved words and identifiers?
What is lex compiler? Illustrate the generation of lexical analyzer with lex.

Two ways to handle reserve word (key word) and identifier

- ① Install the reserved words in the symbol table initially:



return (getToken, installID())

Fig: Transition diagram for ID's and keyword.

'installID' place it in symbol table if it is not already there, and mention as a identifier.

If it is there mention as a reserve word.

'get Token' produce the token name for that identifier

- ② Create Seperate transition diagram for each keyword:

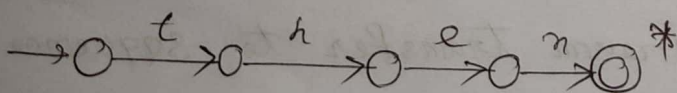
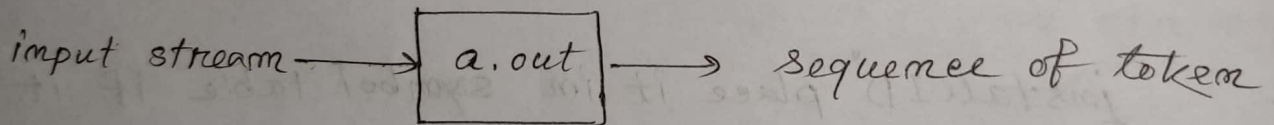
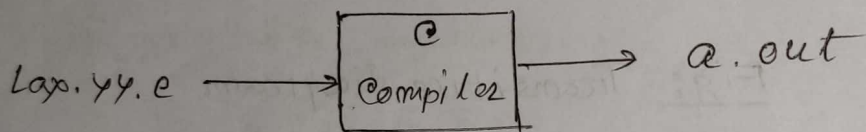
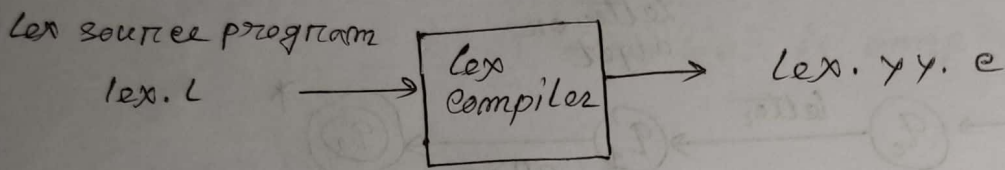


Fig: diagram for keyword 'Then'.

lex compiler:

Lex compiler is a lexical analyzer generator, a tool use to generate lexical analyzer scanner for programming language. Here the tool itself is the lex compiler.

Generation of a lexical analyzer with lex



lex source program (lex.l) is ^{transferred} converted by lex compiler and produce ~~exte~~ lex.yy.c extension. Then c compiler produce a object file (a.out)

Finally the input stream transfer to sequence of tokens

What is regular expression? How to build a regular expression using basis and induction. Describe the language denoted by the regular expression $a(a|b)^*a$.

string of a's and b's that start and end with a.

Ex: a a b a b a a

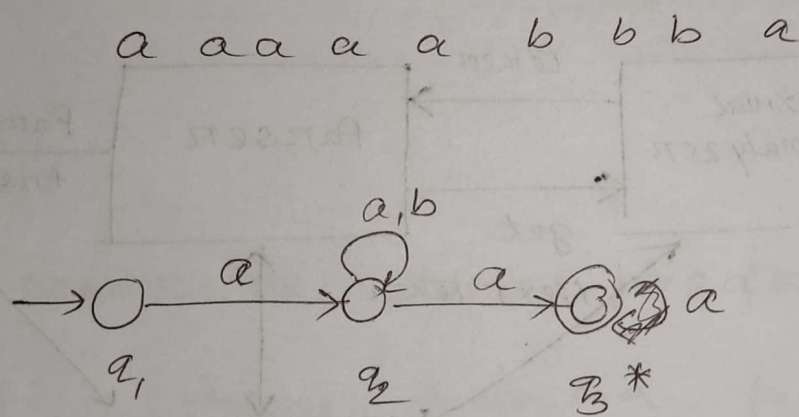


Fig: Transition Diagram

What is the role of parser? Illustrate the position of parser in the compiler model. Define left recursive and non left recursive grammar with example.

Role of parser

1. Construct parse tree
2. Analysis syntactic structure of source code
3. Check correctness, detect error, generate meaningful order.

Position of parser.

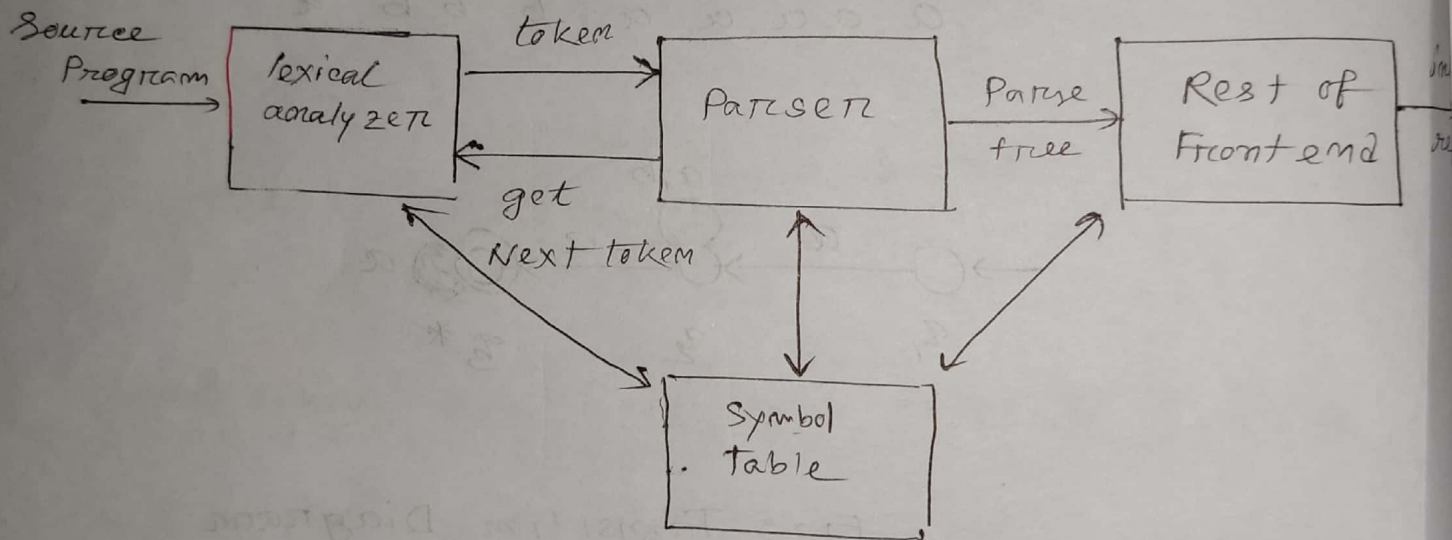


Fig: Position of parser

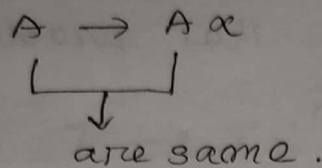
There are three general types of parser for grammar

1. universal
2. Top-down
3. Bottom up

Parser scan from left to right, one symbol at time.

Left recursive:

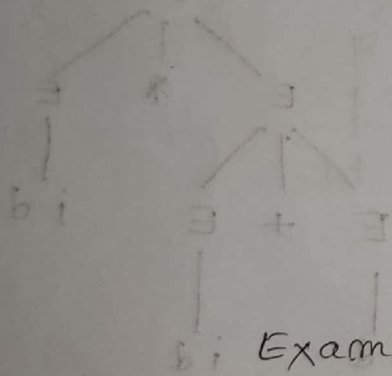
contain a production of the form $A \rightarrow A\alpha$



Example:

$$\begin{array}{l}
 \text{Expr} \Rightarrow \text{Expr} + \text{Term} \\
 \quad \quad \quad | \quad \text{Expr} - \text{Term} \\
 \quad \quad \quad | \quad \text{Term}
 \end{array}$$

Non-left recursive: lacks production of the form $A \Rightarrow A\alpha$ for any non empty string.



Example:

$$\text{Expr} \Rightarrow \text{Term Expr}'$$

$$\text{Expr} \Rightarrow + \text{Term Expr}'$$

$$| - \text{Term Expr}'$$

$$| \epsilon$$

write the significance of the derivational view for the construction of ambiguous parse tree. show two parse tree for "id + id * id".

Illustrate the position of the parser in compiler module. What is ambiguous grammar?

Ambiguous grammar that produces more than one parse tree.

$$E \Rightarrow E + E$$

$$\Rightarrow id + E$$

$$\Rightarrow id + E * E$$

$$\Rightarrow id + id * E$$

$$\Rightarrow id + id * id$$

$$E \Rightarrow E * E$$

$$\Rightarrow E + E * E$$

$$\Rightarrow id + E * E$$

$$\Rightarrow id + id * E$$

$$\Rightarrow id + id * id$$

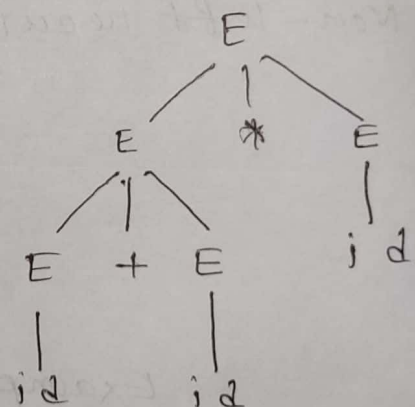
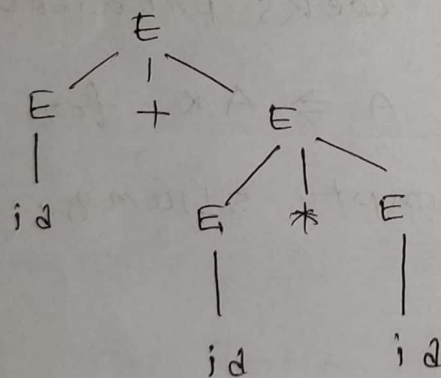


Fig: Two parse tree id + id * id

Significant: derivational view in parsing reveals different ways a sentence can be derived.

Illustrate the sequence of top down parse trees according to grammar (1) for $id + id * id$.

Given,

$$T \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (\epsilon) \mid id.$$

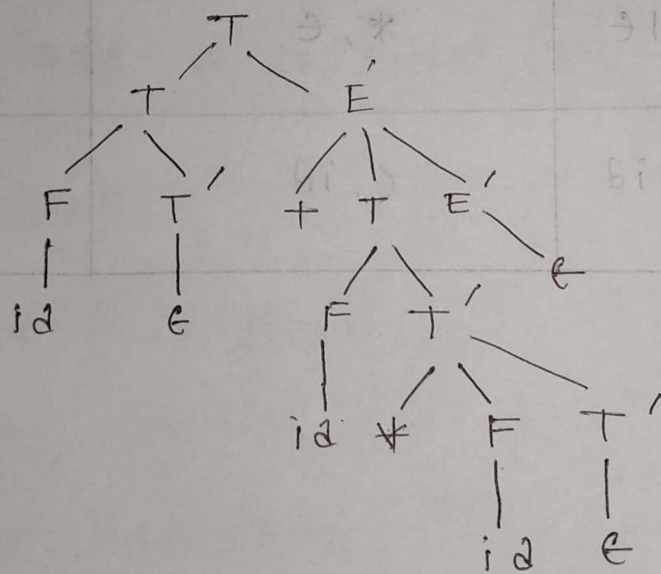


Fig: Parse tree that accept input string $id + id * id$.

First and Follow.

Production	First	Follow
$E \rightarrow TE'$	(, id	\$,)
$E' \rightarrow +TE' \epsilon$	+, ϵ	\$,)
$T \rightarrow FT'$	(, id	+, \$,)
$T' \rightarrow *FT' \epsilon$	*, ϵ	+, \$,)
$F \rightarrow (E) id$	(, id	*, +, \$,)

Explain First and Follow function for constructing of both Top down and bottom-up parser.

Compute (First)

- ① If x is a terminal, then $\text{First}(x) = \{x\}$
- ② If x is non terminal and $x \rightarrow Y_1 Y_2 \dots Y_k$ is the production for some $k > 1$

then the first of x will be First of Y_1

- ③ If $x \rightarrow \epsilon$ is a production then add ϵ to $\text{First}(x)$

Compute (Follow)

- ① Place $\$$ in $\text{follow}(s)$ where s is the first symbol.

- ② If a production $A \rightarrow BC$, then everything in $\text{first}(C)$ except ϵ is the $\text{Follow}(B)$

- ③ If a production is $A \rightarrow B$ then $\text{Follow}(A)$ is the Follow of (B)

Define predictive parser? What is LL(1) Grammar?
How to construct a predictive parsing table?

predictive parser: A predictive parser is a top-down parser that predicts the next production based on the current input symbol and a fixed number of lookahead symbols.

It uses parsing table and avoids back trace.

LL(1) grammar: Predicts the next production based on input and only one lookahead symbol.

LL(1) $\rightarrow$ '1' look-ahead symbol
├── $\rightarrow$ left most derivation.
└── $\rightarrow$ left to right.

Input: Grammar G

Output: Parsing Table M

Method: For each production $A \rightarrow \alpha$

① For each terminal a in $\text{First}(\alpha)$, add $A \rightarrow a$ to

② If ϵ is in $\text{First}(\alpha)$

then for each terminal b in $\text{follow}(A)$ add $A \rightarrow \alpha$ to

If ϵ is in $\text{First}(\alpha)$ and $\$$ is in $\text{follow}(A)$

$\therefore$ add $A \rightarrow \alpha$ to $M[A, \$]$

Parsing Table M.

NonTerminal	Terminal					
	id	+	*	(	)	\$
$E \rightarrow TE'$	$E \rightarrow TE'$			$E \rightarrow TE'$		
$E' \rightarrow +TE' \mid \epsilon$		$E' \rightarrow +TE'$			$E' \rightarrow +TE'$ $E' \rightarrow \epsilon$	$E' \rightarrow +TE'$ $E' \rightarrow \epsilon$
$T \rightarrow FT'$	$T \rightarrow FT'$			$T \rightarrow FT'$		
$T' \rightarrow *FT' \mid \epsilon$		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
$F \rightarrow id \mid (E)$	$F \rightarrow id$			$F \rightarrow id$ $F \rightarrow (E)$		

Distinguish between S-attribute definition and L-attribute definition. Show a dependency graph from the production $E \rightarrow E_1 + T$ and semantic rule $Eval \rightarrow E_1.val + T_1$.

SDD $\rightarrow$ Syntax Directed Translation

Grammar + semantic rule $\rightarrow$ SDD

▣ S Attribute

An SDD is S-attributed if every attribute is synthesized.

S attributed definition can be implemented during bottom up parsing.

It use postorder traversal.

Postorder(N)

{

For (each child c of N From the left)

Postorder(c);

evaluate the attributes associate with node N ;

}

L Attribute :

The second class of SDD is called L attribute definition. edges can go from left to right, but not from right to left.

L attribute must be either

1. Synthesized or

2. Inherited.

Production

$E \rightarrow E_1 + T$

Semantic rule

$E.val = E_1.val + T.val$

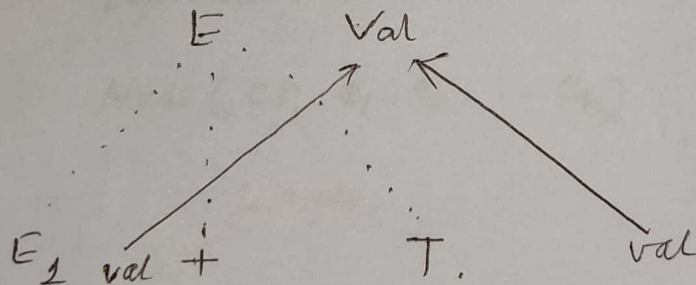


Fig : $E.val$ is synthesized from $E_1.val$ and $T.val$.

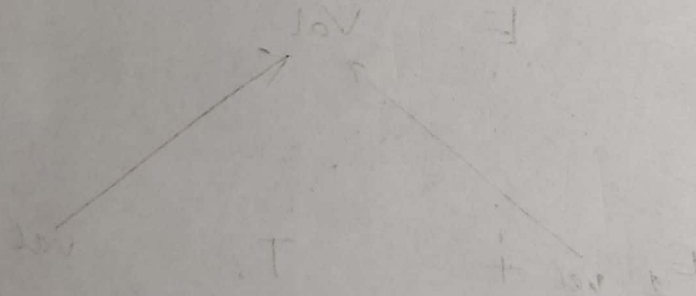
Dependency graph depicts the flow of information among the attribute instances in a particular parse tree.

Synthesized attribute value at N is
computed using the value of at two children
derived E and T.

defⁿ:

SDD $\rightarrow$ SDD is a context free grammar together
with the attribute value and rule.

Annotated parse tree: A parse tree showing
the value of its attribute is called
an annotated parse tree.



Explain the steps in the construction of the syntax tree for $a-4+c$ where the general and simple expressions including production and semantic rule can be considered.

Each node of syntax tree represent construct.

The children of the node represent meaningful components of the construct.

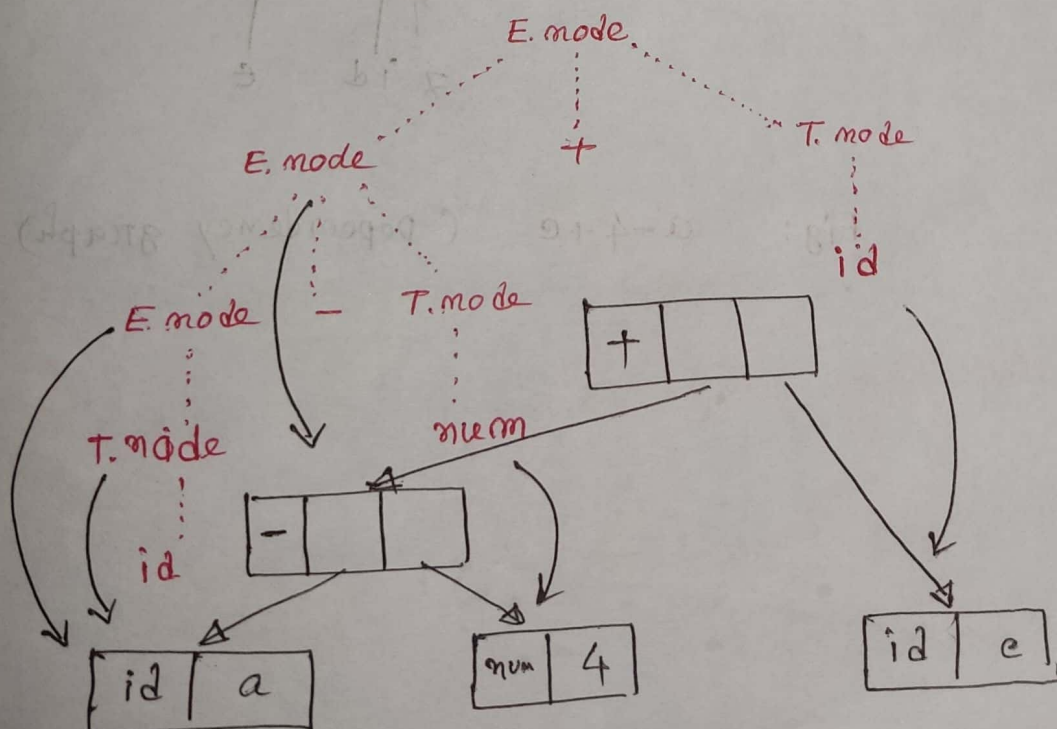
⇒ If the node is leaf: Hold one field category

function leaf (op, value)

⇒ If the node is interior: Hold many field category

function → Node take two or more component

Node (op, $e_1, e_2, \dots, e_k$)



- ① $P_1 = \text{new leaf}(\text{id}, \text{entry a}) ;$
- ② $P_2 = \text{new leaf}(\text{num}, 4) ;$
- ③ $P_3 = \text{new node}('-', P_1, P_2) ;$
- ④ $P_4 = \text{new leaf}(\text{id}, \text{entry c}) ;$
- ⑤ $P_5 = \text{new node}('+', P_3, P_4) ;$

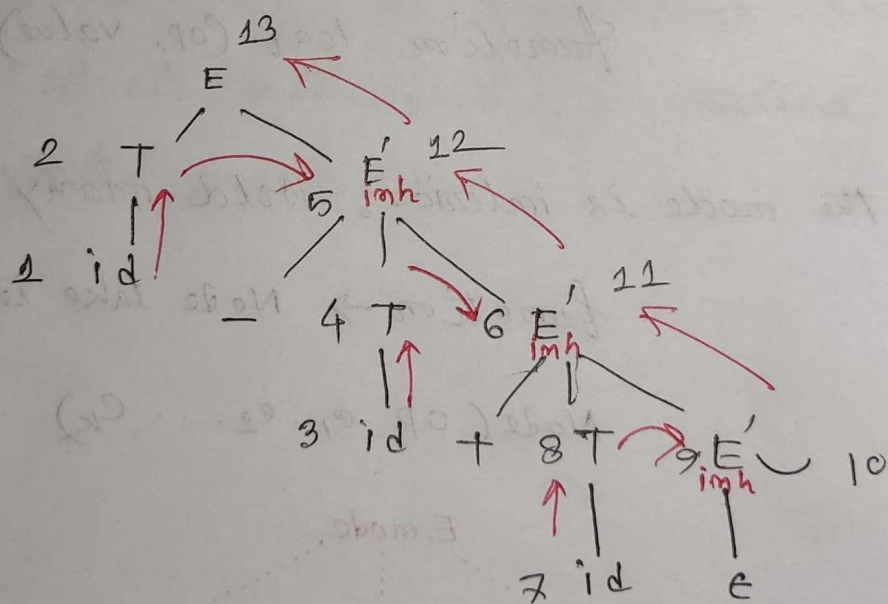


Fig: a-4+c (Dependency graph)

Define directed acyclic graph DAG. Construct a DAG for the expression $a + a * (b - c) + (b - c) * d$. Build a three address code for the expression $a + a * (b - c) + (b - c) * d$.

DAG: A directed acyclic Graph is a graph data structure that consist of nodes connected by directed edges. There are no cycle that means, and each edges has specific direction.

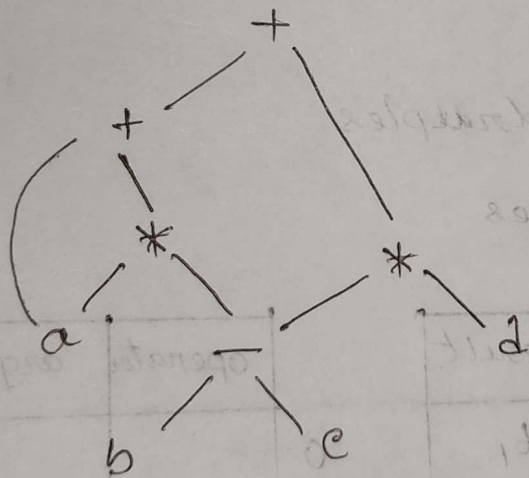


Fig: DAG $(a + a * (b - c) + (b - c) * d)$

❑ New node (op, left, right)

If node returns the existing node otherwise it creates a new node.

Three address code generation.

$$t_1 = b - c$$

$$t_2 = a * t_1$$

$$t_3 = a + t_2$$

$$t_4 = d * t_1$$

$$t_5 = t_3 + t_4$$

alternative representation of Three address code

① Quadruples

② Triples

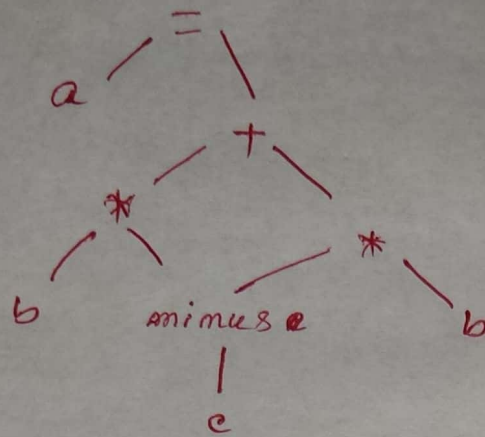
	OP	argument ₁	arg ₂	result
0	minuend -	b	c	t ₁
1	multiplier *	a	t ₁	t ₂
2	+	a	t ₂	t ₃
3	*	d	t ₁	t ₄
4	+	t ₃	t ₄	t ₅
5	+	t₅	+	+

Quadruples

	operator	arg ₁	arg ₂
0			
1			
2			
3			
4			
5			

Triples

$$a = b * -c + b * -c ;$$



$$t_1 = \text{minus } c$$

$$t_2 = b * t_1$$

$$t_3 = \text{minus } c$$

$$t_4 = b * t_3$$

$$t_5 = t_2 + t_4$$

$$a = t_5$$

	operator	args 1	args 2
0	minus	c	
1	*	b	(1)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)